

Bodies of flora: A Three-Stage LLM-Driven Pipeline for Text-to-3D Plant Reconstruction

The project is to bring extinct plant species into life form by creating 3D images from the morphological description.

The created 3D images are not biologically perfect but as they are based on the historical description of the plants.

The plan of action for the project is – There should be use of 3 language models to produce the output, In normal web applications the functions are used when something is triggered by the user and the set of predefined rules will be followed but in this case – the functions are replaced with LLMs to make the web app robust to the texts input that are provided by the users

System Architecture

The pipeline has three main stages plus intermediate processing steps. A user types a plant description, and the system returns a JSON morphology, a 2D image, and a 3D model.

```
User Text -> [Stage 1: LLM] -> JSON -> [Stage 2: SDXL] -> 2D Image -> [Clean] ->
[Stage 3: Hunyuan3D] -> GLB
```

The data flow.

Stage 1 — NLP Morphology Extraction

The first stage converts free-form English text into a structured JSON object that describes the plant's anatomy. In the pipeline, this is handled by calling a large language model through the HuggingFace Inference API.

The LLM receives a system prompt that tells it to behave as a “plant morphology extractor.” It must return a strict JSON with six possible organ blocks: **roots**, **stem**, **leaf**, **flower**, **fruit**, and **seed**. Each block can contain a type, modifications, a general descriptor, and a comparative field (for phrases like “leaves like fern fronds”). The LLM is instructed to never fabricate organs that were not mentioned in the user's text.

Alongside the LLM call, a **rule-based fallback** runs in parallel. This fallback uses regular expressions to detect plant-part keywords and comparative phrases (“like”, “similar to”). If the LLM call fails (network error, API quota, malformed output), the rule-based result is used instead. This dual-path design means Stage 1 never fully fails.

An important sub-task is **scope detection**. If the user says, “show me only the seed,” the system should render just a seed, not a whole plant. The pipeline detects scope keywords like “only” and “just” and sets a target mode flag to “parts” with the specific organs listed. This flag controls what gets rendered in later stages.

Stage 2 — 2D Image Generation

Stage 2 takes the JSON morphology and converts it into a text prompt for an image generation model, then produces a 2D rendering of the plant. In pipeline, the image model is loaded locally using the HuggingFace diffusers library. The default model is FLUX by black forest labs.

The prompt is constructed automatically from the morphology JSON. It starts with “single isolated plant specimen, clean silhouette, studio background, 3d render style” and appends morphology details like organ types and descriptors. A negative prompt excludes common failure modes: blur, noise, text, watermarks, extra objects, and any organs that should not appear.

A critical discovery during development was that **prompt angle matters for 3D quality**. Early versions used prompts like “botanical illustration, front view.” These produced flat, poster-like images that caused the 3D model to reconstruct a flat card instead of a volumetric plant. The pipeline uses “3d render style” prompts with three-quarter angles, which give the 3D model depth cues to work with.

An optional **monochrome mode** converts the generated image to grayscale. This is useful when geometry quality matters more than color, because it removes color noise that can confuse the 3D reconstruction step.

Image Preprocessing (Between Stages 2 and 3)

Before the 2D image is sent to the 3D model, it goes through a cleaning step. This step was added because raw image generator output often contains background artifacts that cause the 3D model to produce garbage geometry.

The preprocessing pipeline works as follows. First, the system samples corner pixels to estimate the background color. Then it computes a difference mask: any pixel whose color is far enough from the background is considered foreground. The foreground region is cropped, scaled to fill 82% of a 1024×1024 canvas, and centered on a clean neutral background (RGB 255, 255, 255). This gives the 3D model a tightly framed, clean input.

Stage 3 — 3D Mesh Generation

Stage 3 takes the cleaned 2D image and produces a 3D triangle mesh. In pipeline, the primary 3D backend is **Hunyuan3D-2.0**, accessed through a HuggingFace Spaces Gradio client.

The “pick best” mode uses a **mesh quality scoring function**. This function loads the output mesh with the trimesh library and scores it based on three factors: face count (more faces generally means more detail), vertex count, and a “detail ratio” (the number of faces in the mesh divided by the number of faces in its convex hull). A mesh that closely matches its convex hull is too smooth; a high detail ratio means the mesh has interesting surface features.

Mesh Post-Processing

After the raw mesh is generated, it goes through a repair step using trimesh: duplicate faces are removed, degenerate (zero-area) faces are removed, unreferenced vertices are cleaned up, and normals are fixed to point outward. The cleaned mesh is exported as a .glb file for display in the Gradio 3D viewer.

Implementation Details

1 Runtime Environment

The entire pipeline runs in a Google Colab notebook with an NVIDIA A100 GPU (40 GB VRAM). The notebook has two cells: Cell 1 installs all dependencies (pinned versions to avoid incompatibilities), and Cell 2 contains the full pipeline code and launches the Gradio web interface. A runtime restart is required between the two cells because Python caches compiled C extensions in memory.

2 Dependency Pinning

One of the most persistent sources of bugs was version incompatibility between Python packages. The pipeline installs exact versions for all critical packages: numpy 1.26.4, scikit-image 0.25.2, diffusers 0.35.1, transformers 4.56.1, gradio 5.49.1, trimesh 4.8.3, among others. This eliminates the class of bugs where updating one library breaks another.

3 Gradio User Interface

The web interface is built with Gradio and provides: a text input box for the plant description, a collapsible configuration panel (model selection, seed, scope overrides, monochrome toggles), output panels for JSON, prompt text, a 2D image gallery, a 3D model viewer, downloadable output files.

4 Space API Discovery

Because HuggingFace Space APIs can change without notice, the pipeline includes an API discovery mechanism. Before calling a Space, it inspects the available endpoints using `client.view_api()`, collects endpoint names, reads parameter metadata, and tries multiple calling conventions (keyword arguments, positional arguments, special Hunyuan) until one succeeds. This makes the pipeline robust to minor API changes in the Spaces.

Cloud Deployment

The pipeline is successfully able to generate in Colab but the same monolithic structure fails/breaks the cloud. The steps before deploying in the cloud(GCP or any other cloud)

1. Containerize Everything (Docker) with 3 different compartments
 - i) NLP Service – running on CPU and calls Groq API
 - ii) Image Service – GPU for FLUX model
 - iii) 3D Service – GPU for Hunyuan3D